

Eliana Lehmann

Deliverable 2: Sustainable Fashion Swapping Microservice

COMPSCI 426

Fall 2025

- **[KG1] Endpoint Definitions:**

File Paths:

user-service/app/usermain.py

item-service/app/itemmain.py

swap-service/app/swapmain.py

Python

User-Service

```
@app.post("/users", response_model=UserResponse,
status_code=status.HTTP_201_CREATED)...

@app.get("/users/{user_id}", response_model=UserResponse)
@app.get("/users", response_model=list[UserResponse])

@app.delete("/users/{user_id}", status_code=status.HTTP_204_NO_CONTENT)
```

Item-Service

```
@app.post("/items", status_code=201, response_model=ItemResponse)

@app.get("/items/{id}", response_model=ItemResponse)

@app.delete("/items/{id}", status_code=204)
```

Swap-Service

```
@app.post("/swaps", status_code=201, response_model=SwapResponse)

@app.get("/swaps/{id}", response_model=SwapResponse)

@app.put("/swaps/{id}/Swapped", response_model=SwapResponse)
```

```
@app.put("/swaps/{id}/reject", response_model=SwapResponse)
```

Justification:

This code snippet defines all the endpoints in my microservice (apart from health checks). These are the access points for managing the service. Each one completes a different request for the microservice.

- **[KG2] Containerization:** The practice of bundling an application's code, configuration, and dependencies into a single, isolated, and portable unit (a container) using technologies like **Docker**.

Python

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
RUN apt-get update && apt-get install -y curl
```

```
COPY app/ ./app/
```

```
CMD ["uvicorn", "app.itemmain:app", "--host", "0.0.0.0", "--port", "8000"]
```

Justification:

This code snippet specifies the steps to build a Docker image for Item Service (each service in my directory has a Dockerfile). It starts from a Python image, sets up the working directory. It installs dependencies, and then sets the final command to run the FastAPI app using Uvicorn.

- **[KG3] Container Orchestration:** The automated management, deployment, scaling, and networking of containers, typically using tools like **Kubernetes** or **Docker Compose** (covered in Weeks 6-8).

Python

```
services:
-----{other code}-----
  swap-service:
    build:
      context: ./swap-service
      dockerfile: Dockerfile

  ports:
    - "8002:8000"

  environment:
    - REDIS_HOST=redis
    - REDIS_PORT=6379
    - ITEM_SERVICE_BASE=http://item-service:8000

  depends_on:
    redis:
      condition: service_healthy
    item-service:
      condition: service_healthy

  restart: unless-stopped

  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 10s
```

Justification:

The docker-compose.yml file is used to orchestrate the multi-service application. The depends_on block ensures the swap-service only starts once its dependencies (redis and item-service) are service_healthy. The healthcheck definition allows Docker Compose to automatically monitor the service's availability, demonstrating automated container orchestration.

- **[KG4] API Gateway:** A single entry point for all clients consuming an API, which handles cross-cutting concerns like security, routing requests to the appropriate microservices, and rate limiting (covered in Week 8).

File Path:

nginx.conf

```
Python
server {
    listen 80;

    location /users/ {
        proxy_pass http://user_service/;
    }

    location /items/ {
        proxy_pass http://item_service/;
    }

    location /swap/ {
        proxy_pass http://swap_service/;
    }

    # default route
    location / {
        return 200 "API Gateway running\n";
    }
}
```

Justification:

Under the api gateway service, which runs nginx, acts as a single endpoint on port 80. It routes requests based on the URL path (/users, /items,/swaps) and uses the proxy_pass to forward the request to the correct service.

- **[KG5] Load Balancing:** Distributing network traffic across multiple servers to ensure no single server is overwhelmed, improving responsiveness and availability (covered in Week 9).

File path:

nginx.conf

```
Python
http {
    upstream user_service {
        server user-service:8000;
    }
    upstream item_service {
        server item-service:8000;
    }
    upstream swap_service {
        server swap-service:8000;
    }
}
```

Justification:

Nginx's upstream configuration is used for load balancing in a scaled environment. It distributes incoming traffic across the defined services. If we were to scale the application, it would ensure that the services and instances of the service are not overloaded.

- **[KG6] Reliability, Scalability, Maintainability:** The three foundational concerns for designing and evaluating any large-scale system: **Reliability** (ensuring the system operates correctly even when parts fail), **Scalability** (the ability to handle increasing load), and **Maintainability** (making it easy for engineers to operate and modify the system).

File path:

`docker-compose.yml`

Python

Reliability

restart: unless-stopped

healthcheck:

test: ["CMD-SHELL", "pg_isready -U user -d user_db"]

interval: 10s

timeout: 5s

retries: 5

Scalability:

http {

upstream user_service {

server user-service:8000;

}

upstream item_service {

server item-service:8000;

}

upstream swap_service {

server swap-service:8000;

}

Maintainability:

fastapi==0.104.1

```

uvicorn==0.24.0
pydantic==2.5.0
pydantic[email]==2.5.0
redis==5.0.1
httpx==0.25.2
python-dotenv==1.0.0
sqlmodel
psycopg2-binary #db driver for postgres

```

Justification:

Reliability: restart : unless-stopped makes sure that the service automatically restarts when it fails, healthcheck is used to monitor the status of the core components of the microservice, ensuring the system operates correctly even with failure.

Scalability: The architecture of the microservice, accessed by API Gateway is scalable as each service is independent of the other, and independently scaled using Docker Compose, ensuring it can handle an increasing number of requests/ load.

Maintainability: The use of small, single-purpose services (User, Item, Swap) and clear, version-controlled dependency management (requirements.txt) makes the codebase easier for engineers to understand, modify, and deploy.

- **[KG7] Service Communication:** communication between and amongst services using HTTP REST techniques or gRPC.

File path:

item-service/app/itemmain.py

Python

```

#service communication validate user service it exists
user_url = f"{USER_SERVICE_BASE}/users/{item.userId}"

```

```

logger.info(f"SERVICE COMMUNICATION: Validating userId {item.userId} via GET
to {user_url}")
try:
    async with httpx.AsyncClient(timeout=3.0) as client:
        user_response = await client.get(user_url)
        if user_response.status_code != 200:
            logger.warning(f"User validation failed for {item.userId}. Status:
{user_response.status_code}")
            raise HTTPException(status_code=404, detail="User not found")
        logger.info(f"User validation successful for {item.userId}.")
except Exception as e:
    logger.error(f"User service unavailable: {e}")
    raise HTTPException(status_code=503, detail="User service unavailable")

```

Justification:

When creating a new item, the Item Service validates that the given user exists. It achieves this by performing a synchronous HTTP GET request to the /users/{item.userId} endpoint. This is a clear example of inter-service communication via HTTP REST.

- **[KG8] Caching Strategies:** Techniques for storing copies of frequently accessed data in a faster-access tier (like Redis) to reduce latency and load on primary data stores (covered in Weeks 9-10).

File path:

`user-service/app/usermain.py`

```

Python
#caching check -> cache hit
cache_key = f"user:{user_id}"
cached_user = redis_client.get(cache_key)

if cached_user:
    logger.info(f"CACHE HIT: User ID: {user_id}")
    return UserResponse.model_validate_json(cached_user)

logger.info(f"CACHE MISS: User ID: {user_id}. Querying DB.")
user = session.exec(select(User).where(User.id == user_id)).first()

```


Justification:

The Get User endpoint implements a read-through cache strategy. In which, it checks Redis for the user data. If a cache hit happens, the response is returned. Reducing latency through accessing the PostgreSQL database. If a cache miss occurs (meaning the requested user data is not in Redis) the data is fetched from the database and written into Redis. This makes requests faster.

- **[KG9] Observability (Logging & Tracing):** The ability to measure the internal state of a system through externally generated data like **logs** (records of events) and **traces** (end-to-end request flows) to aid in debugging and monitoring (covered in Week 10).

File path:

user-service/app/usermain.py (all three services have a structured logging)

```
Python
# Configure basic logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger("user-service")
```

Justification:

This code fragment demonstrates the implementation of logging in this service which is used to measure and understand the internal state of this system. It ensures all critical events that occur with client requests are time stamped and labeled with its location/ service name. This allows for efficient debugging as it makes tracing and monitoring easier.

- **[KG10] Database Per Service Pattern:** An architectural pattern in microservices where each service manages its own private database, ensuring loose coupling and allowing polyglot persistence (covered in Week 11).

File Path:

`docker-compose.yml`

Python

```
user-db:
  image: postgres:15-alpine
  environment:
    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
    POSTGRES_DB: user_db
  volumes:
  --- more code
  -- connects to user-service only
```

```
redis:
  image: redis:7-alpine
  ports:
    - "6379:6379"
  restart: unless-stopped
  volumes:
    - redis-data:/data
  --- more code
  --- connects to item-service, swap-service, and user-service for caching/data
```

Justification:

The User Service uses the dedicated user-db for its data store, and no other service connects to or queries this PostgreSQL instance directly in which it manages its own private database.

The Item and Swap Services use Redis as their primary data store, which is a different type of database of key-value stores for temporary needs.